

✓ CI com GitHub Actions — Bella Tavola 🍷

Parte 2: Testes automatizados com pytest

Como usar este caderno?

Este é o segundo caderno da série de CI com GitHub Actions. Ele assume que você já completou a Parte 1 e tem um pipeline básico rodando no GitHub.

Aqui o trabalho volta a ser majoritariamente em Python — escrevendo testes. Mas os testes precisam rodar no pipeline, então continue com o repositório GitHub aberto ao lado.

Os gabaritos estão nas células colapsadas logo após cada exercício — tente resolver antes de abrir.

O Contexto

Na Parte 1, o pipeline verificava apenas se a API subia — usando `curl` para checar se havia resposta. Isso é útil, mas insuficiente.

Uma API pode subir normalmente e ainda assim retornar dados errados, aceitar inputs que deveriam ser rejeitados, ou quebrar em uma rota específica que o `curl` não testou.

Nesta parte, vamos substituir o `curl` por testes automatizados com `pytest` — e aprender a escrevê-los com qualidade real.

Pré-requisitos

Antes de começar, confirme que você já tem:

- Pipeline da Parte 1 rodando e verde no GitHub Actions
- API do Bella Tavola funcionando localmente
- `httpx` e `pytest` no `requirements.txt`

Se `httpx` não estiver no seu `requirements.txt`, adicione agora — ele é necessário para o `TestClient` do FastAPI funcionar.

O que é essencial, recomendado e desafio?

Essencial: o mínimo que você precisa conseguir fazer para completar a etapa.

Recomendado: amplia a qualidade da solução.

Desafio: aprofunda a solução do ponto de vista de engenharia.

✓ BLOCO 3

Objetivo: Transformar o pipeline do Bloco 2 em um pipeline de CI real — com testes automatizados rodando a cada commit. Ao final deste bloco, você terá um workflow que instala dependências e roda uma suíte de testes estruturada contra a API do Bella Tavola.

✓ Conceitos-chave do Bloco 3

A estrutura de testes que vamos adotar

Neste caderno, vamos colocar todos os testes em uma pasta `tests/` na raiz do projeto e chamá-los explicitamente com `pytest tests/`. Essa convenção facilita a organização e torna o pipeline previsível.

```
bella_tavola/  
├─ main.py  
├─ requirements.txt  
├─ routers/  
├─ models/  
└─ tests/  
    ├─ test_pratos.py  
    ├─ test_bebidas.py  
    └─ test_pedidos.py
```

Sobre o `__init__.py`: em versões modernas do pytest, um arquivo `__init__.py` dentro de `tests/` geralmente não é necessário. Você pode incluí-lo se encontrar erros de importação entre os arquivos de teste, mas não assuma que ele é obrigatório.

Sobre o `requirements.txt` no contexto de CI

No CI, o `requirements.txt` é a única fonte de verdade sobre as dependências do projeto. Se uma biblioteca está instalada na sua máquina mas não está no arquivo, o pipeline vai falhar com `ModuleNotFoundError`.

Para este projeto, mantenha um `requirements.txt` enxuto com as bibliotecas usadas de fato:

```
fastapi
uvicorn
pydantic
pydantic-settings
httpx
pytest
```

Sobre `pip freeze`: o comando `pip freeze > requirements.txt` pode servir como ponto de partida, mas costuma capturar pacotes demais — incluindo ferramentas instaladas no seu ambiente que não fazem parte do projeto. Use com cuidado e revise o arquivo antes de fazer commit.

Como o pytest descobre os testes

O pytest segue convenções automáticas:

- Procura arquivos que começam com `test_` ou terminam com `_test.py`
- Dentro desses arquivos, executa funções que começam com `test_`
- Reporta cada função como um teste individual

```
# pytest vai encontrar e executar esta função:
def test_listar_pratos_retorna_lista():
    ...

# pytest vai ignorar esta:
def verificar_pratos():
    ...
```

O comando pytest no pipeline

```
- name: Rodar testes
  run: pytest tests/ -v --tb=short
```

Flags úteis:

Flag	Efeito
<code>-v</code>	Verbose — mostra o nome de cada teste
<code>--tb=short</code>	Traceback resumido quando um teste falha
<code>-x</code>	Para na primeira falha
<code>-s</code>	Exibe prints e outputs durante a execução

TestClient — testando sem subir um servidor

O `TestClient` do FastAPI permite testar rotas sem precisar de um servidor rodando. Isso torna os testes mais rápidos e mais fáceis de controlar do que usar `curl` com `sleep`.

```
from fastapi.testclient import TestClient
from main import app # ajuste o import para o caminho da sua aplicação

client = TestClient(app)

response = client.get("/pratos")
print(response.status_code) # 200
print(response.json())      # lista de pratos
```

Sobre o import: se sua aplicação não está em `main.py`, ajuste o import de acordo. Por exemplo: `from app.server import app` ou `from api import app`. O importante é importar a instância `FastAPI()` do seu projeto.

✓ Exercício 3.1 — Preparando a estrutura de testes

Nível: Essencial

Conceito: Criar a estrutura mínima para o pytest funcionar no pipeline.

✓ Sua tarefa

1. Crie a pasta `tests/` na raiz do projeto

2. Crie o arquivo `tests/test_saude.py` com o conteúdo abaixo
3. Confirme que `httpx` e `pytest` estão no `requirements.txt`

```
# tests/test_saude.py
def test_pytest_funcionando():
    """Confirma que o pytest encontrou e executou este arquivo."""
    assert 1 + 1 == 2
```

Rode localmente para confirmar:

```
pytest tests/ -v
```

A saída esperada:

```
collected 1 item

tests/test_saude.py::test_pytest_funcionando PASSED [100%]

1 passed in 0.01s
```

Depois, atualize o `ci.yml` para substituir o step de `curl` pelo step de testes:

```
- name: Rodar testes
  run: pytest tests/ -v --tb=short
```

Faça commit e push. Confirme que o pipeline fica verde.

```
# 📝 Anote o que você observou:

# O pipeline passou com o teste mínimo?

# O step de testes apareceu nos logs da aba Actions?

# Quanto tempo o pipeline levou no total?
```

💡 Gabarito

```
# @title
# ci.yml atualizado – step de curl substituído por pytest
#
# name: CI – Bella Tavola
#
# on:
```

```
# push:
#   branches: [main]
# pull_request:
#   branches: [main]
#
# jobs:
#   verificar-api:
#     runs-on: ubuntu-latest
#
#     steps:
#       - name: Baixar o código
#         uses: actions/checkout@v4
#
#       - name: Configurar Python
#         uses: actions/setup-python@v5
#         with:
#           python-version: "3.11"
#
#       - name: Instalar dependências
#         run: |
#           pip install --upgrade pip
#           pip install -r requirements.txt
#
#       - name: Rodar testes
#         run: pytest tests/ -v --tb=short
#
# O TestClient sobe e derruba a aplicação internamente durante os testes,
# sem precisar de uvicorn rodando em background nem de sleep.
# Por isso o step de curl pode ser removido.
```

Exercício 3.2 — Conectando o pipeline ao resultado dos testes

Nível: Essencial

Conceito: Entender como o pytest comunica sucesso ou falha ao pipeline.

Sua tarefa

Adicione um segundo teste ao `test_saude.py` que **vai falhar propositalmente**:

```
def test_que_vai_falhar():
    """Este teste falha de propósito."""
    assert 1 + 1 == 3
```

Faça commit e push. Observe o que acontece na aba Actions.

Como ler o log do pytest: dentro do step "Rodar testes", procure:

1. A linha `collected X items` — mostra quantos testes foram encontrados
2. As linhas com `PASSED` e `FAILED` — uma por teste
3. A seção `FAILURES` — mostra o traceback do que falhou
4. O resumo final — ex: `1 passed, 1 failed in 0.05s`

Depois de observar, **remova o teste que falha** e faça o pipeline voltar ao verde antes de continuar.

```
#  Anote suas observações:

# 1. O pipeline ficou verde ou vermelho?

# 2. Qual foi a mensagem de erro na seção FAILURES?

# 3. O pytest parou no primeiro teste que falhou ou continuou rodando?

# 4. Se o pipeline tem dois jobs com 'needs', e o job de testes falha,
#    o que acontece com o segundo job?
```

Gabarito

```
# @title
# 1. O pipeline ficou vermelho (❌)
#    O step 'Rodar testes' falhou, o que faz o job inteiro falhar.

# 2. A mensagem na seção FAILURES foi algo como:
#    FAILED tests/test_saude.py::test_que_vai_falhar
#    AssertionError: assert 2 == 3

# 3. O pytest NÃO parou – continuou rodando e reportou os dois testes:
#    tests/test_saude.py::test_pytest_funcionando PASSED
#    tests/test_saude.py::test_que_vai_falhar FAILED
#    Para parar na primeira falha, use a flag -x.

# 4. Com 'needs', se o job de testes falha, os jobs dependentes
#    são cancelados automaticamente – aparecem como 'skipped'
#    na interface do GitHub Actions.

# Por que isso funciona:
# O pytest retorna código de saída 0 quando todos os testes passam
# e código diferente de 0 quando algum falha.
# O GitHub Actions interpreta qualquer código diferente de 0
# como falha do step – por isso o pipeline fica vermelho.
```

```
# Você não precisa configurar nada: pytest e Actions já falam  
# a mesma linguagem.
```

✓ Exercício 3.3 — Primeiro teste real da API

Nível: Essencial

Conceito: Usar o `TestClient` do FastAPI para testar rotas.

✓ Referência

```
# tests/test_pratos.py  
from fastapi.testclient import TestClient  
from main import app # ajuste se necessário  
  
client = TestClient(app)  
  
def test_raiz_retorna_nome_restaurante():  
    response = client.get("/")  
    assert response.status_code == 200  
    assert "Bella Tavola" in response.json()["restaurante"]  
  
def test_listar_pratos_retorna_lista():  
    response = client.get("/pratos")  
    assert response.status_code == 200  
    assert isinstance(response.json(), list)  
  
def test_listar_pratos_retorna_pelo_menos_um_prato():  
    response = client.get("/pratos")  
    assert len(response.json()) > 0
```

O que observar: cada `assert` verifica uma coisa específica. Testes pequenos e focados são mais fáceis de depurar — quando falham, você sabe exatamente o que quebrou.

Atenção ao contrato da API: os exercícios deste bloco assumem a API Bella Tavola construída nas Semanas 1 e 2. Se a sua implementação tiver rotas ou campos com nomes diferentes, ajuste os testes para refletir o contrato real da sua API.

Sua tarefa

Crie o arquivo `tests/test_pratos.py` com pelo menos **4 testes** cobrindo:

1. `GET /pratos` retorna status 200 e uma lista
2. `GET /pratos?categoria=pizza` retorna apenas pratos da categoria correta
3. `GET /pratos/1` retorna um prato com os campos esperados (`id`, `nome`, `preco`)
4. `GET /pratos/9999` retorna status 404

Rode localmente antes de fazer push:

```
pytest tests/test_pratos.py -v
```

 Escreva sua solução no arquivo `tests/test_pratos.py`

 Gabarito

```
# @title
# tests/test_pratos.py
from fastapi.testclient import TestClient
from main import app # ajuste se necessário

client = TestClient(app)

def test_listar_pratos_retorna_200():
    response = client.get("/pratos")
    assert response.status_code == 200

def test_listar_pratos_retorna_lista():
    response = client.get("/pratos")
    assert isinstance(response.json(), list)

def test_listar_pratos_retorna_pelo_menos_um_prato():
    response = client.get("/pratos")
    assert len(response.json()) > 0

def test_filtro_por_categoria_retorna_apenas_categoria_correta():
    response = client.get("/pratos?categoria=pizza")
    assert response.status_code == 200
    pratos = response.json()
    for prato in pratos:
        assert prato["categoria"] == "pizza"
```

```
def test_buscar_prato_existente_retorna_campos_esperados():
    response = client.get("/pratos/1")
    assert response.status_code == 200
    prato = response.json()
    assert "id" in prato
    assert "nome" in prato
    assert "preco" in prato

def test_buscar_prato_inexistente_retorna_404():
    response = client.get("/pratos/9999")
    assert response.status_code == 404
```

✓ Exercício 3.4 — Testando criação e validação

Nível: Essencial

Conceito: Testar rotas POST, validação de entrada, status codes de erro.

Testar apenas rotas GET verifica se os dados chegam corretamente. Testar rotas POST verifica se a API aceita o que deve aceitar e rejeita o que deve rejeitar.

✓ Referência

```
def test_criar_prato_retorna_sucesso():
    novo_prato = {
        "nome": "Funghi Trifolati",
        "categoria": "massa",
        "preco": 54.0,
        "disponivel": True
    }
    response = client.post("/pratos", json=novo_prato)
    assert response.status_code in [200, 201]
    assert response.json()["nome"] == "Funghi Trifolati"

def test_criar_prato_com_preco_negativo_retorna_422():
    prato_invalido = {
        "nome": "Prato Inválido",
        "categoria": "pizza",
        "preco": -10.0
    }
```

```
response = client.post("/pratos", json=prato_invalido)
assert response.status_code == 422
```

Sua tarefa

Adicione ao `test_pratos.py` testes para:

1. `POST /pratos` com dados válidos cria o prato e retorna os campos esperados
2. `POST /pratos` com preço negativo retorna 422
3. `POST /pratos` com nome muito curto (menos de 3 caracteres) retorna 422
4. `POST /pratos` com categoria inválida retorna 422
5. O prato criado aparece em `GET /pratos` depois de criado

Atenção — acoplamento temporário: neste bloco, os testes compartilham o estado em memória da API. Dados criados em um teste podem aparecer em outro. É uma limitação real que vamos resolver no Bloco 4 com fixtures. Por enquanto, use nomes únicos nos testes de criação para reduzir a chance de colisão — e anote mentalmente que esse padrão é provisório.

 Adicione os testes ao arquivo tests/test_pratos.py

Gabarito

```
# @title
# Adicionar ao tests/test_pratos.py

def test_criar_prato_valido():
    novo_prato = {
        "nome": "Funghi Trifolati Teste",    # nome único para evitar colisão
        "categoria": "massa",
        "preco": 54.0,
        "disponivel": True
    }
    response = client.post("/pratos", json=novo_prato)
    assert response.status_code in [200, 201]
    dados = response.json()
    assert dados["nome"] == "Funghi Trifolati Teste"
    assert dados["preco"] == 54.0
    assert "id" in dados

def test_criar_prato_com_preco_negativo_retorna_422():
    prato_invalido = {
```

```
        "nome": "Prato Inválido",
        "categoria": "pizza",
        "preco": -10.0
    }
    response = client.post("/pratos", json=prato_invalido)
    assert response.status_code == 422

def test_criar_prato_com_nome_curto_retorna_422():
    prato_invalido = {
        "nome": "AB",    # menos de 3 caracteres
        "categoria": "pizza",
        "preco": 40.0
    }
    response = client.post("/pratos", json=prato_invalido)
    assert response.status_code == 422

def test_criar_prato_com_categoria_invalida_retorna_422():
    prato_invalido = {
        "nome": "Prato Exótico",
        "categoria": "esoterico",
        "preco": 40.0
    }
    response = client.post("/pratos", json=prato_invalido)
    assert response.status_code == 422

def test_prato_criado_aparece_na_listagem():
    # Nome único para não colidir com outros testes ou dados iniciais
    nome_unico = "Tagliatelle Teste XYZ-9871"
    client.post("/pratos", json={
        "nome": nome_unico,
        "categoria": "massa",
        "preco": 68.0
    })
    response = client.get("/pratos")
    nomes = [p["nome"] for p in response.json()]
    assert nome_unico in nomes

# Lembrete: esses testes ainda compartilham estado em memória.
# O Bloco 4 mostrará como lidar com isso de forma mais adequada.
```

✓ Exercício 3.5 — Desafio: cobertura de bebidas e pedidos

Nível: Desafio

Conceito: Aplicação autônoma dos padrões de teste aprendidos.

▼ Sua tarefa

Crie os arquivos `tests/test_bebidas.py` e `tests/test_pedidos.py` seguindo os mesmos padrões do `test_pratos.py`.

Para bebidas, cubra:

- Listagem geral e com filtro por tipo e `alcoolica`
- Busca por ID existente e inexistente
- Criação com dados válidos e inválidos

Para pedidos, cubra:

- Criação de pedido com prato existente e disponível
- Tentativa de pedido com prato inexistente (404)
- Tentativa de pedido com prato indisponível (400)
- O valor total calculado está correto (`preco × quantidade`)

Ao final, rode tudo junto e confirme que o pipeline fica verde:

```
pytest tests/ -v
```

 Implemente nos arquivos `tests/test_bebidas.py` e `tests/test_pedidos.py`

💡 Gabarito — `test_pedidos.py`

```
# @title
# tests/test_pedidos.py
from fastapi.testclient import TestClient
from main import app # ajuste se necessário

client = TestClient(app)

def test_criar_pedido_com_prato_existente():
    payload = {
        "prato_id": 1,
        "quantidade": 2,
        "observacao": "sem cebola"
    }
    response = client.post("/pedidos", json=payload)
    assert response.status_code in [200, 201]
    dados = response.json()
    assert "valor_total" in dados
    assert "nome_prato" in dados
```

```
def test_valor_total_calculado_corretamente():
    prato = client.get("/pratos/1").json()
    preco_unitario = prato["preco"]

    payload = {"prato_id": 1, "quantidade": 3}
    response = client.post("/pedidos", json=payload)
    assert response.status_code in [200, 201]
    assert response.json()["valor_total"] == preco_unitario * 3

def test_criar_pedido_com_prato_inexistente_retorna_404():
    payload = {"prato_id": 9999, "quantidade": 1}
    response = client.post("/pedidos", json=payload)
    assert response.status_code == 404

def test_criar_pedido_com_prato_indisponivel_retorna_400():
    client.put("/pratos/1/disponibilidade", json={"disponivel": False})
    payload = {"prato_id": 1, "quantidade": 1}
    response = client.post("/pedidos", json=payload)
    assert response.status_code == 400
    # Restaura para não afetar outros testes
    client.put("/pratos/1/disponibilidade", json={"disponivel": True})

def test_criar_pedido_com_quantidade_zero_retorna_422():
    payload = {"prato_id": 1, "quantidade": 0}
    response = client.post("/pedidos", json=payload)
    assert response.status_code == 422
```

✓ Checklist do Bloco 3

Antes de seguir para o Bloco 4, confirme que você consegue:

- ☐ Criar a pasta `tests/` e escrever um teste mínimo
- ☐ Usar o `TestClient` para testar rotas sem subir um servidor
- ☐ Testar status codes de sucesso e de erro
- ☐ Testar validação de entrada (422)
- ☐ Ler os logs do pytest na aba Actions e identificar qual teste falhou
- ☐ Rodar `pytest tests/ -v` localmente com todos os testes passando
- ☒ Pipeline verde no GitHub Actions 

✓ BLOCO 4

Objetivo: Escrever testes de qualidade real — previsíveis, isolados e legíveis. Ao final deste bloco, você entenderá por que testes que dependem uns dos outros são problemáticos, como usar fixtures do pytest para melhorar a organização, e o que ainda é necessário para garantir isolamento completo.

✓ Conceitos-chave do Bloco 4

O problema dos testes acoplados

No Bloco 3, observamos que a lista de pratos em memória persiste entre os testes. Isso cria um problema sutil:

```
def test_A():
    client.post("/pratos", json={"nome": "Margherita", ...})
    # cria 1 prato extra na lista

def test_B():
    response = client.get("/pratos")
    assert len(response.json()) == 6    # falha se test_A rodou antes!
```

O `test_B` passa ou falha dependendo da ordem de execução dos testes. Isso é chamado de **acoplamento entre testes** — e é um dos problemas mais difíceis de depurar em suítes grandes, porque a falha parece aleatória.

Fixtures do pytest

Uma **fixture** é uma função que prepara dados ou recursos antes de um teste rodar. O pytest injeta automaticamente as fixtures nos testes que as pedem.

```
import pytest
from fastapi.testclient import TestClient
from main import app

@pytest.fixture
def client():
    """Cria um TestClient para cada teste."""
    return TestClient(app)

def test_listar_pratos(client):    # ← pytest injeta o client aqui
```

```
response = client.get("/pratos")
assert response.status_code == 200
```

O que a fixture de `client` resolve — e o que não resolve

Transformar o `client` em fixture melhora a **organização** do código: você não precisa repetir `client = TestClient(app)` em cada arquivo de teste, e o `client` deixa de ser uma variável global solta.

Mas há um ponto importante: **criar um novo `TestClient` para cada teste não reinicializa o estado interno da aplicação**. Se sua API mantém dados em uma lista Python global, essa lista continua existindo entre os testes — mesmo com a fixture recriando o `TestClient`.

Em outras palavras:

```
@pytest.fixture
def client():
    return TestClient(app)  # novo client, mas mesma lista global de pratos
```

O `test_A` que criou um prato extra ainda vai afetar o `test_B`, mesmo com a fixture. Para eliminar completamente esse acoplamento, seria necessário reinicializar o estado da aplicação a cada teste — o que exige uma refatoração mais profunda da API. Nas estratégias deste bloco, vamos aprender a escrever testes que resistem a esse problema sem precisar refatorar a aplicação.

`conftest.py` — fixtures compartilhadas

Quando vários arquivos de teste precisam das mesmas fixtures, o lugar certo para defini-las é o `conftest.py`:

```
tests/
├─ conftest.py          ← fixtures disponíveis para todos os arquivos
├─ test_pratos.py
├─ test_bebidas.py
└─ test_pedidos.py
```

O pytest descobre o `conftest.py` automaticamente — você não precisa importá-lo em nenhum arquivo.


```
# tests/conftest.py
import pytest
from fastapi.testclient import TestClient
from main import app # ajuste se necessário

@pytest.fixture
def client():
    return TestClient(app)

@pytest.fixture
def prato_valido():
    """Payload de prato válido reutilizável nos testes."""
    return {
        "nome": "Prato de Teste",
        "categoria": "massa",
        "preco": 40.0,
        "disponivel": True
    }
```

```
# tests/test_pratos.py – usando fixtures do conftest
def test_criar_prato(client, prato_valido): # ← ambas injetadas
    response = client.post("/pratos", json=prato_valido)
    assert response.status_code in [200, 201]
```

Testes robustos: verificar comportamento, não estado absoluto

A estratégia mais eficaz para lidar com estado compartilhado sem refatorar a aplicação é escrever testes que verificam **comportamentos relativos**, não contagens ou posições absolutas:

```
# Frágil – assume estado inicial exato:
def test_lista_tem_seis_pratos(client):
    response = client.get("/pratos")
    assert len(response.json()) == 6 # quebra se outro teste criou pratos

# Robusto – verifica comportamento, independente do estado:
def test_prato_criado_aparece_na_lista(client):
    nome_unico = "Prato Único XYZ-4821"
    client.post("/pratos", json={"nome": nome_unico, "categoria": "massa", "preco": 40.0})
    response = client.get("/pratos")
```

```
nomes = [p["nome"] for p in response.json()]
assert nome_unico in nomes
```

A combinação de **fixture de** `client` + **testes que verificam comportamentos relativos** resolve a maioria dos problemas de acoplamento sem precisar reinicializar o estado da aplicação.

Parametrização — testando múltiplos casos com uma função

Quando você quer testar o mesmo comportamento com vários inputs diferentes,

`@pytest.mark.parametrize` evita repetição:

```
import pytest

@pytest.mark.parametrize("preco_invalido", [-1.0, -0.01, -100.0])
def test_preco_invalido_retorna_422(client, preco_invalido):
    prato = {
        "nome": "Prato Teste",
        "categoria": "pizza",
        "preco": preco_invalido
    }
    response = client.post("/pratos", json=prato)
    assert response.status_code == 422
```

Isso gera **três testes** com um único bloco de código. Na saída do pytest:

```
test_pratos.py::test_preco_invalido_retorna_422[-1.0] PASSED
test_pratos.py::test_preco_invalido_retorna_422[-0.01] PASSED
test_pratos.py::test_preco_invalido_retorna_422[-100.0] PASSED
```

Marcadores — organizando e filtrando testes

Você pode marcar testes com etiquetas para rodá-los seletivamente:

```
import pytest

@pytest.mark.smoke
def test_basico_da_api(client):
    ...
```

```
@pytest.mark.validacao
def test_preco_invalido(client):
    ...
```

Para rodar apenas os testes marcados como `smoke`:

```
pytest tests/ -m smoke
```

No pipeline, marcadores permitem separar testes rápidos — que rodam em todo push — de testes mais lentos ou completos, que rodam só no merge para main.

✓ Exercício 4.1 — Criando o conftest.py

Nível: Essencial

Conceito: Centralizar fixtures, eliminar variável global de `client`.

✓ Sua tarefa

1. Crie o arquivo `tests/conftest.py`
2. Mova a lógica de criação do `client` para uma fixture no `conftest.py`
3. Remova a linha `client = TestClient(app)` de cada arquivo de teste
4. Atualize os testes para receber `client` como parâmetro

Rode localmente para confirmar que tudo continua passando:

```
pytest tests/ -v
```

Para refletir: Depois de mover o `client` para uma fixture, um teste que cria um prato ainda pode afetar outro teste? Por que sim ou por que não?

 Escreva o `conftest.py` e atualize os arquivos de teste

 Gabarito

```
# @title
# tests/conftest.py
import pytest
from fastapi.testclient import TestClient
from main import app # ajuste se necessário
```

```
@pytest.fixture
def client():
    """
    Cria um novo TestClient para cada função de teste.
    Fixtures sem escopo explícito têm escopo 'function' por padrão –
    ou seja, são recriadas a cada teste.

    Importante: isso melhora a organização e evita variável global,
    mas NÃO reinicializa o estado interno da aplicação. Se a API
    mantém dados em listas globais, esses dados persistem entre testes.
    A solução é escrever testes que verificam comportamentos relativos
    (veja o Exercício 4.2), não testes que assumem estado absoluto.
    """
    return TestClient(app)


@pytest.fixture
def prato_valido():
    """Payload de prato válido para reutilizar nos testes."""
    return {
        "nome": "Prato de Fixture",
        "categoria": "massa",
        "preco": 45.0,
        "disponivel": True
    }


@pytest.fixture
def bebida_valida():
    """Payload de bebida válida para reutilizar nos testes."""
    return {
        "nome": "Água de Fixture",
        "tipo": "agua",
        "preco": 8.0,
        "alcoolica": False,
        "volume_ml": 500
    }


# tests/test_pratos.py – versão atualizada com fixtures
# (remova a linha: client = TestClient(app))

# def test_listar_pratos_retorna_200(client):          # client injetado pela fix
#     response = client.get("/pratos")
#     assert response.status_code == 200

# def test_criar_prato_valido(client, prato_valido): # duas fixtures injetadas
#     response = client.post("/pratos", json=prato_valido)
#     assert response.status_code in [200, 201]
#     assert response.json()["nome"] == prato_valido["nome"]
```

```
# Para refletir:  
# Sim, um teste que cria um prato ainda pode afetar outro.  
# A fixture recria o TestClient, mas a lista global de pratos  
# continua sendo a mesma entre as chamadas.  
# A solução está no próximo exercício: escrever testes que  
# verificam comportamentos relativos, não contagens absolutas.
```

✓ Exercício 4.2 — Testes robustos vs. frágeis

Nível: Essencial

Conceito: Identificar e reescrever testes que dependem de estado global.

✓ Sua tarefa

Analise os testes abaixo. Cada um tem um problema de fragilidade. Identifique o problema e reescreva de forma mais robusta.

```
# Teste A – frágil  
def test_lista_tem_seis_pratos(client):  
    response = client.get("/pratos")  
    assert len(response.json()) == 6  
  
# Teste B – frágil  
def test_primeiro_prato_e_margherita(client):  
    response = client.get("/pratos")  
    assert response.json()[0]["nome"] == "Margherita"  
  
# Teste C – frágil  
def test_id_do_novo_prato_e_sete(client, prato_valido):  
    response = client.post("/pratos", json=prato_valido)  
    assert response.json()["id"] == 7  
  
# Teste D – frágil  
def test_filtro_pizza_retorna_dois(client):  
    response = client.get("/pratos?categoria=pizza")  
    assert len(response.json()) == 2
```

```
#  Reescreva cada teste de forma mais robusta:  
  
# Teste A reescrito:
```

```
# Teste B reescrito:

# Teste C reescrito:

# Teste D reescrito:
```

Gabarito

```
# @title
# Teste A – problema: assume contagem absoluta de pratos
# Robusto: verifica que a lista não está vazia e tem a estrutura esperada
def test_lista_retorna_pratos_com_estrutura_correta(client):
    response = client.get("/pratos")
    assert response.status_code == 200
    pratos = response.json()
    assert len(pratos) > 0
    assert "id" in pratos[0]
    assert "nome" in pratos[0]
    assert "preco" in pratos[0]

# Teste B – problema: assume posição e nome do primeiro item
# Robusto: verifica que Margherita existe em alguma posição da lista
# (pressupõe que Margherita faz parte dos dados iniciais padrão do Bella Tavola)
# definidos desde a Semana 1 – se seu cardápio inicial for diferente, ajuste)
def test_margherita_esta_no_cardapio(client):
    response = client.get("/pratos")
    nomes = [p["nome"] for p in response.json()]
    assert "Margherita" in nomes

# Teste C – problema: assume qual será o próximo ID gerado
# Robusto: verifica que o ID existe, é inteiro e positivo
def test_novo_prato_recebe_id_valido(client, prato_valido):
    response = client.post("/pratos", json=prato_valido)
    assert response.status_code in [200, 201]
    dados = response.json()
    assert "id" in dados
    assert isinstance(dados["id"], int)
    assert dados["id"] > 0

# Teste D – problema: assume contagem absoluta de pizzas
# Robusto: verifica que o filtro funciona (todos os retornados são pizza)
def test_filtro_categoria_retorna_apenas_categoria_correta(client):
    response = client.get("/pratos?categoria=pizza")
    assert response.status_code == 200
    pratos = response.json()
```

```
for prato in pratos:
    assert prato["categoria"] == "pizza"
```

✓ Exercício 4.3 — Parametrização de casos de erro

Nível: Recomendado

Conceito: `@pytest.mark.parametrize`, testar múltiplos inputs com uma função.

✓ Referência

```
import pytest

@pytest.mark.parametrize("campo,valor_invalido", [
    ("preco", -1.0),
    ("preco", -0.01),
    ("nome", "AB"),
    ("categoria", "esoterico"),
])
def test_campo_invalido_retorna_422(client, campo, valor_invalido):
    prato = {
        "nome": "Prato Válido",
        "categoria": "pizza",
        "preco": 40.0,
        campo: valor_invalido
    }
    response = client.post("/pratos", json=prato)
    assert response.status_code == 422
```

Sua tarefa

Use `@pytest.mark.parametrize` para criar:

1. Um teste parametrizado que verifica que **categorias inválidas** retornam 422. Teste pelo menos 4 categorias inexistentes no Bella Tavola.
2. Um teste parametrizado que verifica que **IDs de pratos que não existem** retornam 404. Use apenas IDs sintaticamente válidos mas inexistentes (ex: `9999`, `123456`).

Atenção: IDs como `-1` ou `0` podem gerar 422 em vez de 404, dependendo de como sua API valida o path parameter. Se quiser

testá-los, faça em um teste separado que aceite ambos os status codes.

3. Um teste parametrizado que verifica que o **filtro de categoria** funciona corretamente para cada categoria válida.

 Escreva os testes parametrizados em tests/test_pratos.py
import pytest

Gabarito

```
# @title
import pytest

@pytest.mark.parametrize("categoria_invalida", [
    "esoterico",
    "fastfood",
    "japonesa",
    "PIZZA",          # case-sensitive – não é igual a 'pizza'
    "massa extra",   # espaço não permitido pelo pattern
])
def test_categoria_invalida_retorna_422(client, categoria_invalida):
    prato = {
        "nome": "Prato Teste",
        "categoria": categoria_invalida,
        "preco": 40.0
    }
    response = client.post("/pratos", json=prato)
    assert response.status_code == 422

# IDs sintaticamente válidos mas inexistentes → espera 404
@pytest.mark.parametrize("id_inexistente", [9999, 123456, 99999])
def test_prato_inexistente_retorna_404(client, id_inexistente):
    response = client.get(f"/pratos/{id_inexistente}")
    assert response.status_code == 404

@pytest.mark.parametrize("categoria_valida", [
    "pizza",
    "massa",
    "sobremesa",
    "entrada",
    "salada",
])
def test_filtro_por_categoria_valida(client, categoria_valida):
    response = client.get(f"/pratos?categoria={categoria_valida}")
```



```
assert response.status_code == 200
pratos = response.json()
for prato in pratos:
    assert prato["categoria"] == categoria_valida
```

✓ Exercício 4.4 — Marcadores e pipeline seletivo

Nível: Recomendado

Conceito: `pytest.mark`, rodar subconjuntos de testes no pipeline.

✓ Sua tarefa

Passo 1: Crie um arquivo `pytest.ini` na raiz do projeto registrando os marcadores:

```
[pytest]
markers =
    smoke: testes básicos que verificam se a API está no ar
    validacao: testes de validação de entrada
    integracao: testes que dependem de recursos externos
```

Passo 2: Marque pelo menos 3 testes com `@pytest.mark.smoke` — escolha os que verificam o comportamento mais essencial da API.

Passo 3: Marque os testes de validação de entrada com `@pytest.mark.validacao`.

Passo 4: Atualize o `ci.yml` para rodar apenas os testes `smoke` em pull requests, e todos os testes em pushes para o `main`.

Para refletir: Qual é a vantagem de ter testes mais rápidos em PRs e testes completos só no merge? Qual é o risco?

```
#  Implemente os marcadores nos arquivos de teste
# e escreva aqui como ficaria o ci.yml com essa lógica:

# ci.yml:
```

 Gabarito

```
# @title
# pytest.ini
# [pytest]
# markers =
```

```
# smoke: testes básicos que verificam se a API está no ar
# validacao: testes de validação de entrada
# integracao: testes que dependem de recursos externos (modelo, banco)

# tests/test_pratos.py – com marcadores
import pytest

# @pytest.mark.smoke
# def test_listar_pratos_retorna_200(client):
#     response = client.get("/pratos")
#     assert response.status_code == 200

# @pytest.mark.smoke
# def test_buscar_prato_existente(client):
#     response = client.get("/pratos/1")
#     assert response.status_code == 200

# @pytest.mark.smoke
# def test_criar_prato_valido(client, prato_valido):
#     response = client.post("/pratos", json=prato_valido)
#     assert response.status_code in [200, 201]

# @pytest.mark.validacao
# @pytest.mark.parametrize("preco_invalido", [-1.0, -0.01, -100.0])
# def test_preco_invalido_retorna_422(client, preco_invalido):
#     prato = {"nome": "Prato Teste", "categoria": "pizza", "preco": preco_invalido}
#     response = client.post("/pratos", json=prato)
#     assert response.status_code == 422

# ci.yml com testes seletivos por evento
#
# name: CI – Bella Tavola
#
# on:
#   push:
#     branches: [main]
#   pull_request:
#     branches: [main]
#
# jobs:
#   testes-smoke:
#     runs-on: ubuntu-latest
#     steps:
#       - uses: actions/checkout@v4
#       - uses: actions/setup-python@v5
```

```
#         with:
#         python-version: "3.11"
#         - run: pip install --upgrade pip && pip install -r requirements.txt
#         - name: Rodar testes smoke
#         run: pytest tests/ -m smoke -v --tb=short
#
# testes-completos:
#   runs-on: ubuntu-latest
#   needs: testes-smoke
#   # Só roda em pushes para o main, não em pull requests
#   if: github.event_name == 'push' && github.ref == 'refs/heads/main'
#   steps:
#     - uses: actions/checkout@v4
#     - uses: actions/setup-python@v5
#       with:
#         python-version: "3.11"
#     - run: pip install --upgrade pip && pip install -r requirements.txt
#     - name: Rodar todos os testes
#     run: pytest tests/ -v --tb=short
#
# Por que cada job repete checkout e instalação?
# Cada job roda em uma máquina virtual separada e isolada.
# Não há estado compartilhado entre jobs – cada um começa do zero.
#
# Vantagem da separação:
# PRs ficam com feedback mais rápido (só smoke).
# Risco: um bug nos testes não-smoke pode chegar ao main
# sem ser detectado antes do merge.
# Os testes smoke devem cobrir os caminhos mais críticos da API.
```

✓ Exercício 4.5 — Desafio: testando o contrato da API 🎯

Nível: Desafio

Conceito: Testes de contrato — verificar que a API respeita o schema prometido.

Um **teste de contrato** verifica não apenas o status code, mas o formato completo da resposta. Se a API parar de retornar um campo prometido, o teste detecta imediatamente.

✓ Referência

```
def test_contrato_prato_output(client):
    response = client.get("/pratos/1")
    assert response.status_code == 200
    prato = response.json()
```

```
# Verifica que todos os campos esperados existem
campos_obrigatorios = {"id", "nome", "categoria", "preco", "disponivel"}
assert campos_obrigatorios.issubset(prato.keys())

# Verifica os tipos de cada campo
assert isinstance(prato["id"], int)
assert isinstance(prato["nome"], str)
assert isinstance(prato["preco"], float)
assert isinstance(prato["disponivel"], bool)

# Verifica restrições de negócio
assert prato["preco"] > 0
assert len(prato["nome"]) >= 3
```

Sua tarefa

Crie o arquivo `tests/test_contratos.py` com testes de contrato para:

1. O schema de resposta de `GET /pratos/{id}`
2. O schema de resposta de `POST /pratos`
3. O schema de resposta de erro 404 — campo `detail` ou `erro`, sendo uma string não vazia
4. O schema de resposta de erro 422 — verificar que `detail` é uma lista, e que cada item tem pelo menos os campos `loc` e `msg`

Dica para o item 4: o FastAPI retorna erros 422 com `detail` como uma lista de objetos. Cada objeto tem `loc` (localização do erro), `msg` (mensagem) e `type`. Se você implementou um exception handler customizado, o formato pode ser diferente — adapte o teste ao formato real da sua API.

Importante: use a fixture `client` do `conftest.py` — não redeclare aqui.

```
#  Implemente em tests/test_contratos.py
# Use a fixture 'client' do conftest.py — não redeclare aqui.
```

Gabarito

```
# @title
# tests/test_contratos.py
```

```
# A fixture 'client' vem do conftest.py – não é necessário redeclará-la aqui.

def test_contrato_get_prato(client):
    response = client.get("/pratos/1")
    assert response.status_code == 200
    prato = response.json()

    campos_obrigatorios = {"id", "nome", "categoria", "preco", "disponivel"}
    assert campos_obrigatorios.issubset(prato.keys())

    assert isinstance(prato["id"], int)
    assert isinstance(prato["nome"], str)
    assert isinstance(prato["categoria"], str)
    assert isinstance(prato["preco"], (int, float))
    assert isinstance(prato["disponivel"], bool)
    assert prato["preco"] > 0
    assert len(prato["nome"]) >= 3

def test_contrato_post_prato(client):
    novo = {
        "nome": "Prato Contrato Teste",
        "categoria": "massa",
        "preco": 45.0
    }
    response = client.post("/pratos", json=novo)
    assert response.status_code in [200, 201]
    prato = response.json()

    assert "id" in prato
    assert isinstance(prato["id"], int)
    assert prato["nome"] == "Prato Contrato Teste"

    if "criado_em" in prato:
        assert isinstance(prato["criado_em"], str)
        assert len(prato["criado_em"]) > 0

def test_contrato_erro_404(client):
    response = client.get("/pratos/9999")
    assert response.status_code == 404
    corpo = response.json()

    # A API pode usar 'detail' (padrão FastAPI) ou 'erro' (handler customizado)
    assert "detail" in corpo or "erro" in corpo
    mensagem = corpo.get("detail") or corpo.get("erro")
    assert isinstance(mensagem, str)
    assert len(mensagem) > 0
```

```
def test_contrato_erro_422(client):
    # Payload com múltiplos erros para garantir estrutura rica na resposta
    response = client.post("/pratos", json={"nome": "X", "preco": -1})
    assert response.status_code == 422
    corpo = response.json()

    # Formato padrão do FastAPI: {"detail": [...]}
    # Handler customizado pode usar: {"detalhes": [...]}
    erros = corpo.get("detail") or corpo.get("detalhes")
    assert erros is not None, "Resposta 422 deve conter 'detail' ou 'detalhes'"
    assert isinstance(erros, list), "Os erros devem ser uma lista"
    assert len(erros) > 0, "Deve haver pelo menos um erro reportado"

    # Verifica a estrutura de cada erro (formato padrão FastAPI)
    if "detail" in corpo:
        for erro in erros:
            assert "loc" in erro, "Cada erro deve ter 'loc' (localização)"
            assert "msg" in erro, "Cada erro deve ter 'msg' (mensagem)"
            assert isinstance(erro["loc"], list)
            assert isinstance(erro["msg"], str)
            assert len(erro["msg"]) > 0
```

✓ Checklist do Bloco 4

Antes de seguir para a Parte 3, confirme que você consegue:

- ☐ Criar e usar fixtures no `conftest.py`
- ☐ Explicar o que a fixture de `client` resolve e o que ela não resolve sozinha
- ☐ Reescrever testes frágeis para verificar comportamentos relativos
- ☐ Usar `@pytest.mark.parametrize` para testar múltiplos casos
- ☐ Criar marcadores e usá-los para filtrar testes no pipeline
- ☐ Explicar por que cada job repete checkout e instalação de dependências
- ☐ Rodar `pytest tests/ -v` com todos os testes passando ✓
- ☐ Pipeline no GitHub Actions verde com a suíte completa ✓

✓ Checklist de competências — Parte 2

Marque o que você consegue fazer ao final deste caderno:

Bloco 3 — Testes com pytest

- ☐ Criar testes com `TestClient` do FastAPI
- ☐ Testar status codes, estrutura de resposta e validação de entrada

- ☐ Ler os logs do pytest na aba Actions e identificar qual teste falhou
- ☐ Integrar testes ao pipeline com `pytest tests/ -v`

Bloco 4 — Qualidade dos testes

- ☐ Usar fixtures e `conftest.py` para organizar os testes
- ☐ Explicar por que a fixture de `client` melhora organização mas não garante isolamento de estado
- ☐ Escrever testes que verificam comportamentos relativos, não estado absoluto
- ☐ Usar parametrização para cobrir múltiplos casos com uma função
- ☐ Usar marcadores para separar testes smoke, validação e integração
- ☐ Configurar o pipeline para rodar subconjuntos de testes por evento